

오늘의 작업기록 — 2026년 7월 18일

m106 clone 실행 — 레포 VPS 안착

(수련회 둘째날, 어제 확보한 명령을 실제로 실행한 마디)

한눈에

항목	내용
상황	수련회 둘째날 — 시간 거의 없음, 잔디 필요
판단	7/11 원칙 — 빈 커밋 대신 실질 전진(어제 clone 명령 → 오늘 실행)
한 일	<code>git clone</code> 실행 → <code>codecrafters-redis-java</code> 레포 VPS 안착 ✓
오늘의 마찰	clone 주소 찾기 — <code>app.codecrafters.io</code> (보는 곳) vs <code>git.codecrafters.io</code> (받는 곳) 착각
미리 파악 2	① <code>pom.xml</code> = 빌드는 Maven (내일 mvn 없으면 <code>sdk install maven</code>) ② <code>Main.java</code> 위치 확인
안 한 것	정찰(README·Main.java 내용) — 일부러 (정찰+코드는 같은 호흡, 내일 뒀)
잔디	이 작업기록 파일의 logs 커밋
다음	복귀 후 세션: 정찰 → before 실패 → Stage 1 주석 해제 (브라우저 UI 없이 시작)

1. 오늘의 마찰 — "보는 곳"과 "받는 곳"은 다르다

오늘 유일하게 막힌 곳: clone 주소를 찾는 것. 브라우저 주소창에 뜬 URL을 clone 명령에 넣으려 반복 시도했는데 —

주소	정체	용도
<code>app.codecrafters.io/courses/redis/introduction</code>	보는 곳 — 사람이 브라우저로 읽는 코스 페이지	clone 불가
<code>git.codecrafters.io/1e6aca72db41e76f</code>	받는 곳 — git이 코드를 가져오는 저장소	<code>git clone</code> 대상

핵심: 브라우저 주소창(`app...`)이 아니라, 코스 화면 본문의 **"Clone the repository"** 복불칸(`git...`) 이 답이었다. 웹에서 보는 페이지 URL과 git 저장소 URL은 별개다 — GitHub도 마찬가지로(`github.com/...`을 보다가 clone은 `.git` 주소), CodeCrafters는 도메인 자체가 `app/git`으로 갈려서 더 뚜렷하다. 이게 오늘 얻은 구분이다. ("Test CLI connection" 복불칸도 화면에 있었지만 오늘 안 씀 — 그건 Stage 1 통과 후 실제 서버에 붙어보는 단계라 내일 뒀.)

2. clone 실행 + 구조 파악

실행

```
cd ~ && git clone https://git.codecrafters.io/1e6aca72db41e76f codecrafters-redis-java
# Cloning into 'codecrafters-redis-java'... done.
# (명령에 cd codecrafters-redis-java까지 붙어 있어 풀더 안 자동 진입)
ls ~ | grep -i redis    # → codecrafters-redis-java
```

구조 (ls -R로 지도만, 내용은 안 엮)

```
codecrafters-redis-java/
├── README.md           ← 내일 첫 정찰
├── pom.xml             ← Maven 빌드파일
├── your_program.sh     ← 로컬 실행 스크립트
├── codecrafters.yml    ← 코스 설정
└── src/main/java/Main.java ← 내일 주석 해제 대상
```

구조만 보고 미리 잡힌 것 둘: - `pom.xml` 존재 → 빌드는 **Maven**이다. 내일 걸릴 수 있는 지점이 예고됐다 — mvn이 없으면 `sdk install maven` 한 줄(SDKMAN 이미 있음). `ledger-api`는 Gradle이었는데 이 코스는 Maven — 첫 Maven 프로젝트다. - **Main.java 위치 확인**(`src/main/java/`) — 내일 정찰+주석 해제 대상. 표준 자바 레이아웃이라 낯익다.

정찰을 일부러 안 한 이유

README·Main.java **내용**은 열지 않았다. 이 코스는 "Study and uncomment" (읽으며 주석 해제) 방식이라, 정찰(무슨 코드인지 읽기)과 실행(해제하고 통과)이 **같은 호흡**이어야 한다. 오늘 정찰만 떼면 내일 그 내용을 다시 읽어야 하니 이득이 없다. 구조(ls)는 지도라 봐도 되지만, 내용은 내일 코드와 붙여서 본다.

3. 각주 — serenity 공존 정리 (7/17 저녁, 학습 로드맵 밖)

7/17 저녁 VPS 홈을 열었을 때 "파일이 섞였다"고 느껴 정리했다. 기록으로만 남긴다(로드맵 밖 인프라 작업이라 본문 아님):

- **실제로 안 섞여 있었다.** 학습 프로젝트(backend-journey, ledger-api, mini-ledger 등)와 트레이딩 봇(serenity/, systemd 상시가동)이 각자 폴더로 이미 격리됨. 문제는 홈 루트에 뜬 파편 3개(setup_serenity_v3.sh, backend-journey.zip, code-server.log)와 정체 불명이던 _archive(→ 과거 8-K 트레이딩 잔해로 확인).
- **안전 절차:** WorkingDirectory=/root/serenity 확인 → 파편이 봇과 무관함을 검증한 뒤 이동. rm이 아니라 _archive로 mv(복구 가능). 봇은 is-active = active 유지, 무중단.
- **교훈:** 상시 가동 서비스가 걸린 환경에선 "확인 먼저, 이동은 나중". 봇이 어느 경로를 몰고 있는지 모른 채 mv하면 죽는다.

4. 로드맵 위치

- **m106:** 계정(7/17) → clone 실행(오늘) 완료. 남은 건 정찰 → before 실패 → Stage 1 주석 해제 → done 판정.
- **7월 잔여(13일):** m106 마무리 → m107 월말 점검. 복귀 후 첫 세션이 m106 완주.
- 8월 WAL 이유 3개 유지: 원자성 한계(7/5) · 영속성 부재(7/12) · 롤백 불가(7/13).

5. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. app.codecrafters.io와 git.codecrafters.io의 차이는? clone에 쓰는 건 어느 쪽이고 왜인가?
2. clone하기 전에도 이미 알 수 있었던 "내일 빌드 도구"는 무엇이고, 그걸 어떤 파일로 알았나?
3. 오늘 README · Main.java 내용을 일부러 안 연 이유는? 이 코스 방식과 어떤 관계인가?
4. (인프라) 상시 가동 서비스가 있는 홈을 정리할 때 "확인 먼저"란 구체적으로 뭘 확인하는 것인가? rm 대신 mv를 쓴 이유는?

6. 회고

착수 조각의 두 번째 마디를 났다. 어제가 계정+clone 명령 확보였고, 오늘이 그 실행이다 — 둘 다 5분짜리지만, 이렇게 쪼갠 덕에 수련회 3일 내내 잔디가 이어지면서 m106이 조금씩 전진했다. 통째 이월이었으면 3일이 통째로 비었을 것이다.

오늘 얻은 건 작지만 실전에서 자주 걸리는 구분이다 — "보는 주소"와 "받는 주소". 브라우저에 뜬 URL을 그대로 git에 넣으려다 두 번 헛짚었는데, 코스 화면 본문의 복불칸(git 도메인)이 답이었다. GitHub에서도 같은 원리지만 CodeCrafters는 app/git으로 도메인이 갈려서 더 선명하게 배웠다. 그리고 clone만 하고도 pom.xml 하나로 내일 빌드가 Maven이란 걸 미리 알았다 — 구조를 먼저 보는 것의 값이다.

내일(복귀 후)은 드디어 코드다. Main.java를 열어 읽으며 주석을 풀고, accept() 한 줄에서 "기다림"의 정체를 처음 본다. 6379를 내 손으로 여는 것 — 7/6에 마법 목록 ①로만 적었던 그것의 아래층. 레포는 이미 안착했으니, 남은 건 여는 일뿐이다.

한 줄: 어제 확보한 clone 명령을 실행해 Redis 레포를 VPS에 안착 — "보는 주소(app) vs 받는 주소(git)" 구분을 배웠고, pom.xml로 내일 빌드가 Maven임을 미리 파악. 정찰은 코드와 같은 호흡이라 내일로.